Structured Programming Methods

Published by:

NCC Education Limited

The Towers, Towers Business Park

Wilmslow Road, Didsbury

Manchester M20 2EZ

UK

Chapter 2 Variables, Control, Structures and Calculations

Introduction

Every programming language, like languages such as English, has rules which must be followed. The English language has words which must be used in a grammatical way to have meaning. For example the nouns computer, mouse, screen and data are nouns and click, enter and use, are verbs. These are examples of the syntax of the language. The sentence "You can use the mouse to click on the screen to enter data into the computer" is meaningful. When learning the English language you might start with words such as nouns and verbs, moving on to adjectives and start to build phrases. Sentences are then developed, moving on to paragraphs where the sentences within the paragraph are of the same topic. It is easier for a student of English to write a paragraph than it is to write an essay or report.

Definition: syntax (syntax rules)

The rules defining the legal sequences of symbolic elements in a language. The syntax rules define the form of the various constructs in the language, but say nothing about the meaning of these constructs.

Definition: language constructs

A syntactic structure or set of structures in a language to express a particular class of operations. The term is often used as a synonym for control structure. We can relate a programming language to the English language. The building blocks are not nouns and verbs but reserved words and keywords which have a special meaning in the programming language. Examples of keywords are PRINT, INPUT and examples of reserved words are IF, REPEAT. Program statements or instructions are similar to sentences and procedures, functions and subroutines are similar to paragraphs in that they contain the instructions to perform a particular task. These terms will be addressed in more detail in chapter 4.

Since this workbook is concerned with concepts associated with programming rather than a particular programming language you will be using pseudo code throughout the workbook. Pseudo code is one of the tools that can be used to write a preliminary plan which can be developed into a computer program. It is not a standard language, although programmers often use terms within it that closely resemble the actual language to be used. Its purpose is to describe the algorithm (the method of solving the problem) in a form that can easily be understood and then translated into the actual programming code required.

The syntax of the language to be used and the fine detail of the program are ignored until writing the source code, i.e. the program, which will be compiled (translated into binary digits that the computer can use) later.

You will also be drawing diagrams to represent the algorithm. The diagrams are similar to flowcharting, although the official standards are not used. Flowcharting was used extensively in the 1960s before pseudo code and other charting techniques were available. The diagrams used have been chosen as compatible with more modern techniques of charting used later in the course for the object oriented programming languages.

The first problem you will consider is to input three numbers from the keyboard of a personal computer, to add them together and to output the result. This can be thought of as a more general problem in that data is accepted, a process is performed on the data and the results are returned to the user or retained for use at a later stage.

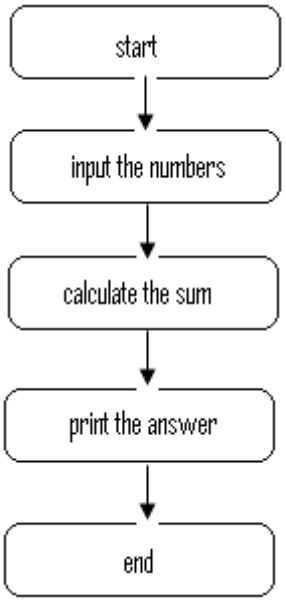


Figure 2.1: Solution in the form of a diagram

There must be a method of accepting data and storing it in the computers memory. The computer program also needs to know where the data is stored in memory so that it can process the data. Programming languages use variables for this purpose.

Variable Types and Names

Variables are used to store data in the computer's memory (RAM) during the time that the program is running. At this stage we will consider three attributes associated with variables. They are: -

1. the address in memory where the data is stored;
2. the actual data stored which can change during the execution of the program;
3. the name of the variable or identifier.

In the example to calculate the sum of three numbers you would need four variables. Programmers can choose the names of the variables, with certain limitations which we will address shortly. It is good practice to think of names which are meaningful in the context of the program; it will help you to understand the coding.

In this case we could use number1, number2, number3 and sum. The pseudo code could be written as: -

```
ACCEPT number1, number2, number3
sum:= number1 + number2 + number3
PRINT sum
end program
```

When you declare number1 as a variable, an address in memory is allocated to it and the number input through the keyboard is stored in this memory location. Every time the identifier number1 is used in the program, this memory location would be accessed and thus the data stored in it can be read and manipulated. The programmer uses the identifier to gain access to the data, the generated program code deals with the connection between the variable and the memory location.

It was mentioned earlier that there are certain limitations on the names you can use for variables. The name chosen must conform to certain rules so that the computer you are using to build your program does not receive ambiguous instructions. First, this means that you cannot use any of the words that make up the programming language. These words are called reserved words and have a special meaning in the programming language. All programming languages have a unique list of reserved words and you should make sure you are familiar with the ones used in the programming language you will be learning.

Punctuation marks and spaces may also have a special meaning within the grammar or syntax of the computer language you are learning, so these cannot be used as part of a variable name, either. You should make sure that you know how to create valid variable names in the programming language you will be using. In the pseudo code you have used so far, the words ACCEPT and PRINT can be thought of as reserve words.

Identifiers are usually written as text although there are differences between programming languages in terms of the additional characters that can be used. In the pseudo code we will use the underline symbol to combine text to provide a meaningful name. E.g. average_student for the average mark for a student. In some languages this would be written as AverageStudent.

Definition: pseudo code

Another name for pseudo language.

Definition: pseudo language

Another name for pseudo code. A form of representation used to provide an outline description of the specification for a software module. Pseudo languages contain a mixture of natural language expressions embedded in syntactic structures taken from programming language (such as IF ...THEN ...ELSE). The formality of the definition varies from ad hoc (e.g. defined within the project team) to being sufficiently formal to enable automatic parsing and syntax checking (e.g. supported by a CASE (Computer Aided Software Engineering) tool). Pseudo languages are not intended to be executed by computer; they must be interpreted by people.

Definition: variable

A unit of storage that can be modified during program execution, usually by assignment or read operations. A variable is generally denoted by an identifier or by a name.

Definition: assignment statement

A fundamental statement of all programming languages (except declarative languages) that assigns a new value to a variable. The typical form in Algol-like languages is variable:= expression where := is read as “becomes”; the symbol suggests a left-pointing arrow to signify the conveyance of a value to the variable on the left. Other languages (particularly BASIC, C and FORTRAN) use = as the assignment operator, e.g. a = b + c.

This leads to problems in expressing the concept of equality. BASIC, being an unsophisticated language, is able to use = for both purposes; C uses = = for equality and FORTRAN uses EQ.

Definition: identifier

A string of characters used to identify (or name) some element of a program. The kind of element that can be named depends on the programming language; it may be a variable, a data structure, a procedure, a statement, a higher-level unit, or the program itself.

Definition: name

A notation for indicating an entity in a program or system. (The word can also be used as a verb.) The kinds of entity that can be named depend on the context, and include variables, data objects, functions, types, and procedures (in programming languages), nodes, stations, and processes (in a data communication network), files, directories, devices (in operating systems), etc. The name denotes the entity, independently of its physical location or address. Names are used for long-term stability (e.g. when specifying a node in a computer program) or for their ease of use by humans (who recognize the name more readily than an address). Names are converted to addresses by a process of name lookup.

In many languages and systems, a name must be a simple identifier, usually a textual string. In more advanced languages, a name may be composed from several elementary components according to the rule of the language.

Definition: reserved word

A word that has a specific role in the context in which it occurs, and therefore cannot be used for other purposes. For example, in many programming languages the words ‘IF’, ‘THEN’, ‘ELSE’ are used to organize the presentation of the written form of statements (between ‘THEN’ and ‘ELSE’ and following ‘ELSE’) whose execution is governed by the value of the Boolean expression between ‘IF’ and ‘THEN’. The use of IF, THEN or

ELSE as identifiers is thus not permitted in these languages since they are reserved words.

Definition: keyword

A symbol in a programming language that has a special meaning for the compiler or interpreter. For example, keywords in BASIC include IF, THEN, PRINT. The keywords guide the analysis of the language, and in a simple language each keyword causes activation of a specific routine in the language processor.

Definition: compiler

A program that translates high-level language into absolute code, or sometimes into assembly language. The input to the compiler (the source code) is a description of an algorithm or program in a problem-oriented language; its output (the object code) is an equivalent description of the algorithm in a machine-oriented language.

Definition: interpreter

A language processor that analyses a line of code and then carries out the specified action, rather than producing a machine-code translation to be executed later.

Variable Types

The compiler or interpreter needs to know what type of data is being used because it will have a different way of dealing with whole numbers, fractional numbers and characters. As you will see later, the type of data being used has a great influence on program design. The different types of data are represented as follows: -

Whole numbers (e.g. 1, 678, 45) – Integer

Fractional numbers (e.g. 3.678, 789.3) – Real or Float

Letters (e.g. a, f, z, 4) – Character

Names (e.g. Dr. H. Kelly) – String

FALSE or TRUE, 0 OR 1 – Boolean (only two values allowed)

English postal codes contain letters, numbers and a space (which the computer treats as a blank character code rather than as an absent character) e.g. BD19 3FG. The computer would use this data as a string because it has a blank character in it. Numbers are only declared as integer or real if they are going to be used in calculations.

Exercise 2.1

Choose an appropriate data type for variables to be used in each of the following situations:

a) For a metric measurement.

b) To count a number of people.

c) To calculate the total of a grocery bill.

d) For a choice between 4 letters as in a multi-choice examination exercise.

e) An international telephone number e.g. (01444) 123456.

f) A catalogue code e.g. AZ1234.

Many programming languages insist on a variables definition at the beginning of the program. This emphasizes the importance of data in deciding the structure of the program. It is good practice to identify the variables used and the types in the pseudo code. It is also good practice to describe any printed or displayed output, rather than just printing a number. You must remember that the output and input to the program are the human-computer interface and as such must be meaningful and helpful to the user. The pseudo code is now: -

```
Program start
    Use variables: number1, number2, number3, sum OF TYPE Integer
    DISPLAY "Enter three numbers"
    ACCEPT number1, number2, number3
    sum:= number1 + number2 + number3
    PRINT "The sum is " sum
 end program
```

Although variables are used to hold data which can change throughout the program, the data stored is always under the control of the program. Some variables may have data entered at the beginning of the program and the data stays the same throughout the running of the program (e.g. tax rate). You may wonder why a variable is used in this case, but consider the implications if a program performed calculations such as: tax_amount := price*17.5/100. The program code would have to be changed if the tax rate changed from 17.5%.

Data items which have the same value throughout the program are termed constants. As with variables, the computer needs a name for each constant and to know its data type so that it can be manipulated as required during the program. Constants and variables obey the same rules and can be compared or used in arithmetic expressions provided they are of the same data type or of compatible types for the specific operation, e.g. integer and floating point. If 17.5 had been used in the above statement, we would call this a literal, not a constant. If a literal is other than a number; e.g. the heading of a report could contain “student name” and “grade”, then just as quotation marks have been used to help you distinguish the text as different to the words in the sentence, so must you enclose the text in quotation marks in the pseudo code. For example DISPLAY “Enter the student’s name”.

Definition: constant

1. A quantity or data item whose value does not change.
2. A value that is determined by its denotation, i.e. a literal.

Definition: literal

A word or symbol in a program that stands for itself rather than as a name for something else, i.e. an object whose value is determined by its denotation. Numbers are literals; if other symbols are used as literals it is necessary to use some form of quoting mechanism to distinguish then from variables.

Precedence

In the pseudo code statement; sum := number1 + number2 + number3, a calculation is performed and the result assigned or copied into the location identified by the variable sum. Note that this means that the original data stored in the variable sum would be overwritten. Computer languages provide a range of operators for arithmetic. The signs that will be used in our pseudo code are: -

+ meaning add

– meaning subtract

* meaning multiply

/ meaning divide

For the moment, this is all we will use in our examples. The order in which a calculation is evaluated is very important since the end result can differ according to which operation is given precedence, i.e. performed before another. We shall do a simple example to illustrate the potential problem. The expression a:= 4 + 3*2 could be worked out in two different orders.

1) Add the 4 to the 3 and then multiply the result by 2	4+3=7	7*2=14
2) Multiply the 3 by the 2 and then add 4 to the result	3*2=6	6+4=10

One method of working has resulted in an answer of 14, the other an answer of 10. You may have encountered this problem during your early mathematics lessons.

Having rules for the order of precedence of the arithmetic operations avoids this ambiguity. The way expressions are worked out by a computer are very similar to the rules in mathematics, and are:

do operations in ***brackets first***;

then evaluate any ***exponentiation*** (e.g. finding the cube of a number);

then any ***multiplication*** or ***division operation***;

and lastly ***addition*** and ***subtraction***.

Thus the correct answer for the expression a:= 4 + 3*2 is 10. If you wanted the addition to be performed first, you would have to write the expression as a:= (4 + 3)*2 and then the correct answer would be 14.

Study Note

It is important to understand that the code in the example performed the calculation first and then assigned the value to the variable named a. An assignment is NOT the same as the logical operator equals =, which we use in mathematics. To help you remember this difference our pseudo code uses the sign :=. An assignment such as total := total + score means that the number currently stored in the variable named total is added to the number currently stored in the variable score. The result is then assigned to the variable total, thus overwriting the original value of total.

Exercise 2.4

Write the pseudo code to solve the following problem: -

> Input two decimal numbers and calculate and print the sum and product of the two numbers. Provide text prompts for the input data and text descriptions of the output provided. Remember to identify variable names and type.

Control Structure 1 – Sequence

There are three main types of instructions used in programming, sequence, selection and loops. The problems met so far have all included instructions which have to be carried out in sequence. This is the easiest type of program control but it is very important that the instructions are in the right order as the computer will obey the instructions implicitly in the order in which they are written.

Definition: control structure

A syntactic form in a language to express flow of control. Common control structures are: IF ... THEN ... ELSE, WHILE ... DO, REPEAT ... UNTIL, CASE.

Control Structure 2 – Selection

There are many occasions where a program is required to include a range of alternative actions. Let's say that you were asked to write a program in which one number (a constant) is divided by another (a variable). The contents of this variable can change throughout the program and it is possible that the variable could contain a zero, particularly if the data was entered via the keyboard and the operator had made a mistake.

As it is not possible to divide by zero, the calculation should not be performed on this data, so the condition would have to be tested for. You would need to instruct the computer "if a zero is entered then print 'invalid entry', and do not do the division calculation". This can be illustrated in the form of a flowchart. Flowcharts were used extensively by programmers before the advent of pseudo code. However, they are still a useful tool to help programmers work out the logic of the sequence of instructions needed, particularly where conditions occur. Note that the results of the test are placed on the lines exiting from the decision box. It is best to write the condition in a form where the two choices are yes or no (i.e. TRUE or FALSE) as this matches the IF ... THEN ... ELSE statement.

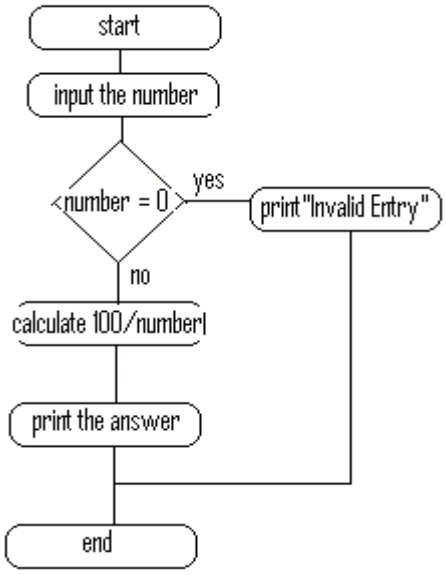


Figure 2.2: Checking for dividing by zero

Another example of selection might be when we need to take action based on user choice, as in a menu of options where the user may input alternative choices and, in consequence, the program must act on these choices.

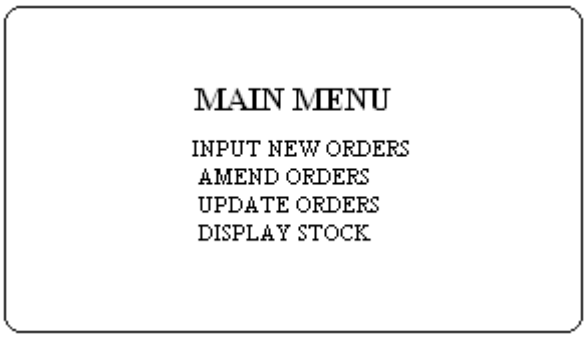


Figure 2.3: An example of a menu showing user choices

All computer languages therefore provide some means of selection. Usually, this is in the form of an IF statement and our pseudo code is no exception to this. We shall use the IF statement together with logical operators (such as equal, less than) to test for true or false as shown below: -

IF a = b THEN PRINT "a is equal to b" where the '=' sign is the logical operator.

The action to be taken is preceded by the word THEN, and is only taken if the test result is true. The logical operators used in our pseudo code are:

=	is equal to
>	is greater than
<	is less than
>=	is greater than or equal to
<=	is less than or equal to
<>	is not equal to

Hence IF a <= b translates to 'if a is less than or equal to b'. Let us now illustrate this with a practical example. In this example the problem is to offer the user a menu of choices which will allow the input of two numbers and the calculation of the sum, difference or

product of the numbers. Figure 2.4 shows the prompts provided for the user, and the data that the user keys in. Choose one of the following: -

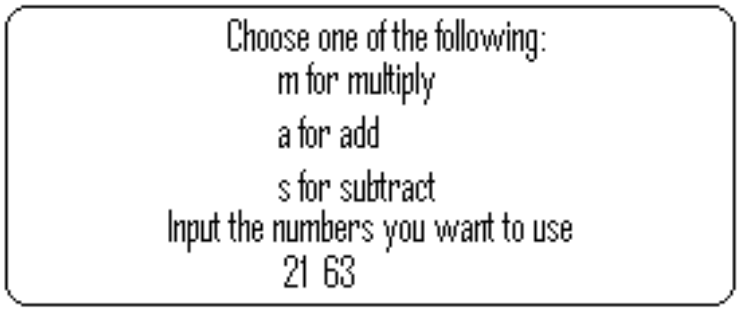


Figure 2.4 Screen display when the program is running

The pseudo code solution is: -

```
Use variables: choice OF TYPE Character
               answer, number1, number2 OF TYPE Integer
DISPLAY "Choose one of the following:"
DISPLAY "m for multiply"
DISPLAY "a for add"
DISPLAY "s for subtract"
ACCEPT choice
DISPLAY "Input the numbers you want to use"
ACCEPT number1, number2
IF choice = "m" THEN
    answer:= number1 * number2
ENDIF
IF choice = "a" THEN
    answer:= number1 + number2
ENDIF
IF choice = "s" THEN
    answer:= number1 - number2
ENDIF
DISPLAY answer
```

end program

Exercise 2.5

Amend the solution to also include divide as a choice. Identify the changes to the pseudo code you will need to make. Remember to ask yourself whether the type of variables will stay the same.

Conditional statements can be quite complex. Some examples in English are: -

> "If I've an assignment to finish off I'll have to do it, otherwise I'll go and see the football match with you next week."
>
> "When the alarm goes off get straight out of bed, unless its a weekend in which case you can stay in bed a bit longer."

When we talk to each other we can include quite complex conditional sentences but we are still able to understand the meaning. Computers are not quite so intelligent and we need to be very precise in the way that we provide instructions.

You already know that in an IF statement the instructions following the THEN part of the IF statement are executed if the condition is TRUE, but an IF statement on its own is often not the neatest way of solving a problem. A more elegant and comprehensive set of conditions can be created by adding an ELSE statement to the IF statement. The IF ... THEN ... ELSE statement is used to deal with a situation such as "A person is paid at top value for category 1 work otherwise his pay is at normal rates." In this example there is some processing to do when the expression is FALSE. This leads to the following logical statement: IF the work is category 1, THEN pay-rate is top ELSE pay-rate is normal. Therefore, in pseudo code we write: -

```
IF work = cat1 THEN
    p_rate := "top"
ELSE
    p_rate := "normal"
```

ENDIF

Exercise 2.6

Amend your pseudo code solution to exercise 2.5 to use the IF ... THEN ....ELSE statement rather than a series of IF....THEN statements.

Exercice 2.7

Write the pseudo code for a program to calculate the wages of a salesman according to the following rules. The wage is calculated at a rate of 15% of sales. If the salesman has been with the company more than three years, he has a loyalty bonus of 10% of his calculated wage. HINTS: What output is required? What processing is needed? What input data is required?

Exercise 2.8

Write the pseudo code for a program to input a category of insurance and display details of the type of insurance available. Use the IF...THEN...ELSE statements.

Category	**Type of Insurance**
U	Insurance not available
A	Insurance is doubled
B	Insurance is normal
M	Insurance is medically independent

Any other category entered is invalid.

Definition: IF THEN ELSE statement

The most basic conditional construct in a programming language, allowing a selection between two alternatives, depending on the truth or falsity of a given condition. Most languages also provide an IF ... THEN construct to allow conditional execution of a single statement or group of statements. Primitive languages, such as BASIC in its original form, restrict the facility to a conditional transfer of control, e.g. “IF A = 0 THEN 330”

which is reminiscent of the conditional jump provided in the order code of every Central Processing Unit.

The CASE Statement

Repeating the IF ... THEN ... ELSE statements a number of times can be somewhat confusing. A cleaner (and easier to follow) construct is to use a selection method that specifically covers the set of alternative conditions that are needed. In our pseudo code this will be called a CASE statement.

The CASE statement is frequently used for coding the choice between items in lists such as those found in screen menus. The solution to exercise 2.8, using the CASE statement rather than IF ... THEN ... ELSE statements, is shown below. Note that the amount of code required with the CASE statement is much less to solve the same problem.

Solution to Exercise 2.8 using a CASE statement:

```
Program start
  Use variables: category OF TYPE Character
  ACCEPT category
  DO CASE of category
    CASE category = "U"
      PRINT "Insurance := not available"
    CASE category = "A"
      PRINT "Insurance := double"
    CASE category = "B"
      PRINT "Insurance := normal"
    CASE category = "M"
      PRINT "Insurance := medically dependent"
    OTHERWISE PRINT "entry is invalid"
  ENDCASE
end of program
```

Study Note

Note that the last alternative statement in the CASE statement is OTHERWISE. This statement will be executed if no other case has been selected as TRUE. Some programming languages, such as C, use fall-through CASE structures and serious errors can arise if the OTHERWISE case is not included. You are advised to always include the OTHERWISE case in every CASE statement.

Definition: CASE statement

A conditional control structure that appears in most modern programming languages and allows a selection to be made between several sets of program statements; the choice is dependent on the value of some expression. The case statement is a more general structure than the IF…THEN…ELSE statement, which allows a choice between only two sets of statements.

Logical operations

The conditions set in the IF...THEN statements have been concerned with operands being equal but there are many occasions when we will need to extend the conditions that are to be tested. Often these conditions are linked in a logical way, and as such are called logical operations. In English we might say things like: “If I had the time and the money I would go on holiday”. The and means that both conditions must be true before we take an action. We might also say “I am happy to go to the theatre or the cinema.” The logical link this time is or. Conditions in IF…THEN statements are linked in the same way. Conditions linked with an AND only result in an action when all conditions are true.

For example: IF a > b AND a > c THEN display “a is the largest”.

Conditions linked with an OR lead to an action when either condition is true. Let us look at a practical example to illustrate this. The problem is to input an examination mark and test it for the award of a grade. The mark is a whole number between 1 and 100. Grades are awarded according to the following criteria: -

>= 80 distinction

>= 60 and < 80 merit

>= 40 and < 60 pass

< 40 fail

Pseudo code solution: -

```
Program start
  Use variables: mark OF TYPE Integer
  PRINT "Enter a mark"
  ACCEPT mark
  IF mark >= 80 AND mark <= 100 THEN
      DISPLAY "distinction"
  ELSE IF mark >= 60 AND mark < 80 THEN
      DISPLAY "merit"
  ELSE IF mark >= 40 AND mark < 60 THEN
      DISPLAY "pass"
  ELSE IF mark >= 0 AND mark < 40 THEN
      DISPLAY "fail"
  ELSE
      DISPLAY "Invalid entry for a mark"
  ENDIF
end program
```

Exercise 2.9

Using the pseudo code solution provided above, complete the diagram shown in figure 2.6 to show the solution to the problem of inputting an examination mark and testing it for the award of a grade of distinction, merit, pass or fail. The diagram has been completed for the grades of distinction and merit. You will be completing the algorithm by adding the part for a grade of pass or fail.

Definition: logic operation

An operation on logical values, producing a Boolean result (see also Boolean algebra).

The operations are denoted by symbols known as operators. In general there are 16 logic operations over one or two operands; they include AND, OR, NOT, NAND, NOR, exclusive-OR, and equivalence.

Definition: logical type

(Boolean type) A data type comprising the logical values TRUE and FALSE, with legal operations restricted to logic operations.

Definition: logical value

(Boolean value) Either of the two values TRUE and FALSE that indicate a truth value. Although a single bit is the most obvious computer storage structure that can be applied to logical data, larger units of store, such as byte, are frequently used in practice since they can be addressed distinctly.

Exercise 2.10

(a) Amend the code in the previous pseudo code solution to the problem of inputting marks and allocating grades. Include the additional criteria that if a mark is <40 and >=30 the grade is a referral. But this time use IF...THEN...ELSE statements instead of the four IF...AND...THEN statements.
(b) Draw a flowchart diagram to show your solution.

Control Structure 3 – Loops

The power of a computer lies in its ability to do things time and time again, very quickly, without becoming tired, bored, or inaccurate. The sequence of instructions which is repeated is called a loop. They can also be referred to as iterations, but strictly speaking an iteration is more complex than a simple loop.

The instructions needed to process all the payroll records on a file and then print pay slips are an example of a loop: -

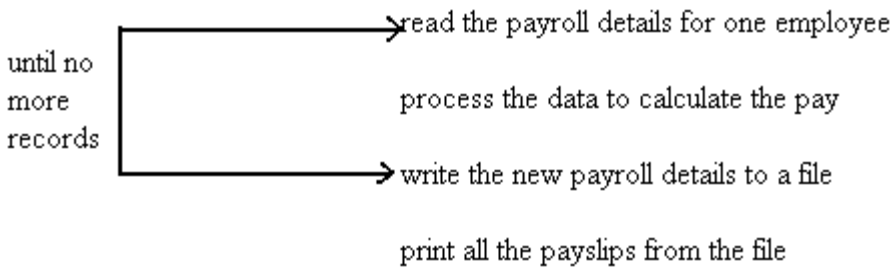


Figure 2.7: Running the payroll to illustrate 'looping'

The instructions are in the form of a loop and these instructions will be repeated for every record on the payroll file. The loop will terminate when end-of-file is reached. This means that the instruction after the loop will then be executed, in this example the pay slips would then be printed. An iteration is a repetition of a sequence of instructions where the results from one pass of the loop are used as input to the instructions in the next pass of the loop. An example of an iteration would be if, during the running of the payroll loop above, a running total was kept of e.g. tax to be paid.

Each time through the loop, the tax for the current employee would be added to the old current total which would then be used as input in the following loop for the next employee. The instructions which make up the loop have two main elements: -

i. a sequence of instructions to be performed each time the loop is executed;

ii. an indication of when to finish executing the loop. This part will obviously contain a logical test as a certain condition will need to be checked.
This checking can be done at different stages of the loop, e.g. at the beginning or at the end, depending on the type of instruction used to control the iteration.

There are three constructs for loops in our pseudo code: -

i. REPEAT UNTIL which tests at the end of a block of code, so the sequence of instructions is always executed at least once.

ii. WHILE which tests at the start of a block of code so it is possible that

the instructions in the loop may never be executed.

iii. FOR loop which is controlled by a count given from known conditions.

Definition: loop

A sequence of instructions that is repeated until a prescribed condition, such as agreement with a data element or completion of a count, is satisfied.

REPEAT an action or block of actions UNTIL (a true condition occurs)

This type of loop is often used for processing input data which will have a coded item at the end of the data to indicate that the end of the data has been reached.

For example, percentage examination marks could be terminated by a number outside the range of 0 to 100. Consider a program segment to keep allowing entry of a number in the range 0 to 100. Pseudo-code Solution: -

```
Use variables: number of type Integer
REPEAT
  DISPLAY "Enter a number between 0 and 100"
  ACCEPT number
UNTIL number <0 or number > 100
end program
```

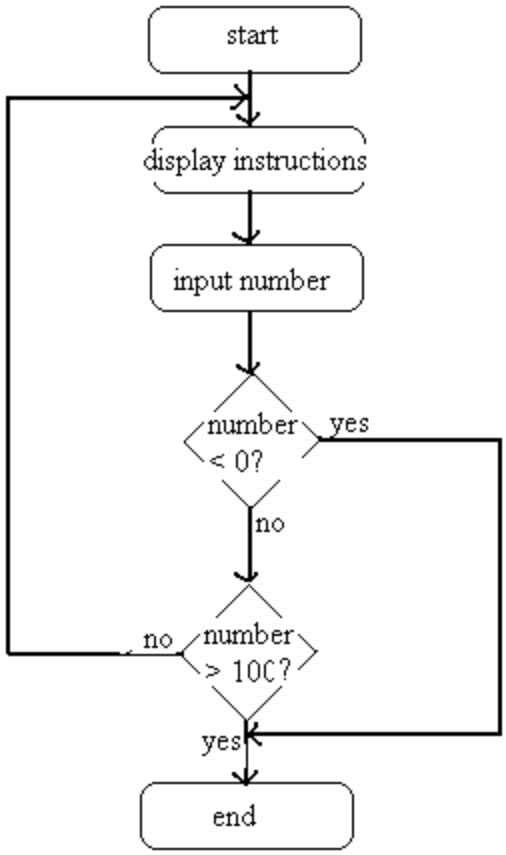


Figure 2.8: Algorithm for only accepting a number between 1 and 100

Another common use is in selection from a menu of options, as in the following example. A survey has been carried out to discover the most popular sport. The results will be typed into the computer for analysis. A pseudo code solution is: -

Use variables: letter OF TYPE Character

athletics, swimming, football, badminton OF TYPE Integer

REPEAT

DISPLAY "Type in the letter chosen or Q to finish"

DISPLAY "A : Athletics"

DISPLAY "S : Swimming"

DISPLAY "F : Football"

DISPLAY "D : Badminton"

DISPLAY "Q : end data:"

```
    ACCEPT letter
    IF letter = 'A'
      THEN athletics := athletics + 1
    ENDIF
    IF letter = 'S'
      THEN swimming := swimming + 1
    ENDIF
    IF letter = 'F'
      THEN football := football + 1
    ENDIF
    IF letter = 'B'
      THEN badminton := badminton + 1
    ENDIF
  UNTIL letter = 'Q'
  DISPLAY "Athletics scored " athletics "votes"
  DISPLAY "Swimming scored " swimming "votes"
  DISPLAY "Football scored " football "votes"
  DISPLAY "Badminton scored " badminton "votes"
  end of program
```

Study Note

Note the statement swimming := swimming + 1. To remind you, this means that the current data stored in the variable swimming is added to 1 and the answer is placed back in the variable swimming. It is assumed that the variable swimming will contain zero at the beginning of the program. When this type of statement is in a loop, and the loop can be executed several times, it is important that the variable contains a zero at the beginning of each count. It is good practice to make all running totals zero before the beginning of the block of statements that contain the statement to add to the running total.

Exercise 2.11

What difference would it make to the way the program worked if the REPEAT instruction was moved to immediately before the instruction ACCEPT letter?

Exercise 2.12

A number of student results are to be entered using the keyboard and the results are to be printed. Each student will have a variable number of results, terminating in a value of 999. Although the number of marks for each student is variable there will usually be between 1 and 5, so there will be no problem printing the marks for each student on one line. The student names will terminate with XXXX. Write the pseudo code to calculate the total score for each student and print the student name, each of the student's marks and the total. An example of output might be: -

Student Name	**Marks**	**Total**
John Smith	34 56 72	162

where the data typed at the keyboard for two students would be: -

John Smith, 34, 56, 72, 999, William Holland, 67, 43, 12, 34, 999, XXXX.

WHILE a true condition occurs DO an action or block of actions

In the second type of iteration we test for the terminating condition at the beginning of the code block. WHILE (a true condition) STATEMENT or STATEMENT BLOCK concluded with ENDWHILE. A statement BLOCK means a group of statements needed to complete a particular process required. The statements would be indented to indicate that they are 'together'. For example: A program segment to display each character typed at a keyboard until the character q is entered.

```
Use variables: letter OF TYPE Character
ACCEPT "letter"
WHILE letter <> 'q'
   DISPLAY "the character you typed is ", letter
```

ACCEPT letter

ENDWHILE

Study Note

In this example the statement block contains two statements which are always executed together, a DISPLAY statement and an ACCEPT statement.

Exercise 2.13

(a) Draw a flowchart diagram to show this algorithm.

(b) Why are there two ACCEPT "letter" instructions in the solution?

Definition: DO-WHILE loop

A form of programming loop in which the condition for termination (continuation) is computed each time around the loop. There are several variants on this basic idea. For example, Pascal has: -

```
WHILE <condition> DO
    BEGIN
        <statements>
    END
```

and also;

```
REPEAT
    <statement>
UNTIL <condition>
```

The first is a WHILE loop and the second is a REPEAT UNTIL loop. Apart from the obvious difference that the first specifies a continuation condition while the second specifies a termination condition, there is a more significant difference. The WHILE loop is a zero-trip loop, i.e. the body will not be executed at all if the condition is false the first time around. In contrast, the body of a REPEAT-UNTIL loop must be obeyed at least

once. Similar constructs are found in most languages, though there are many syntactic variations. See also DO loop.

Exercise 2.14

Consider the flowchart shown in Figure 2.9 of a part of a program to accept a letter over the keyboard, and then answer the following exercises: -

a) When this part of the program terminates, what data is stored in the variable letter?

b) How many times will the loop be executed when the user tries to enter the data, p s e u d o q o d e?

c) Describe the effect of entering the data U u D d O o Q q U u O o.

d) Complete the following table to indicate the contents of the variable letter and the displayed character for each pass of the loop when the letters p s e u d o q are typed.

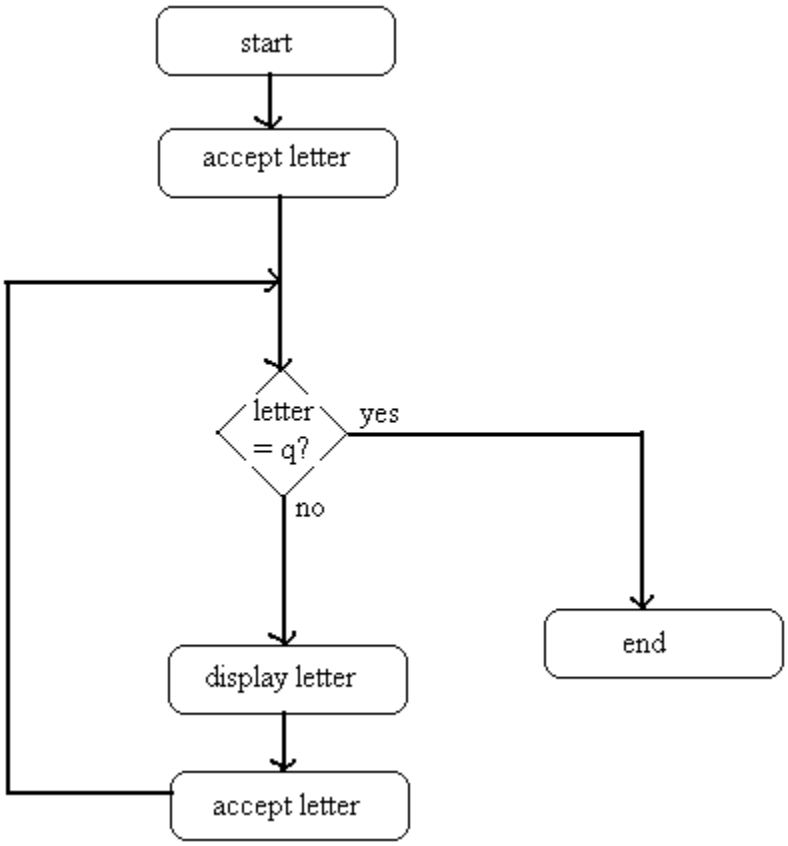


Figure 2.9: Flowchart for algorithm of typing letters over the keyboard

REPEAT an action or block of actions FOR a number of times

The third type of loop, which we shall use when the number of repetitions is known in advance, is a FOR loop. This, in its simplest form, uses two values of a variable, the starting and the final condition for the action.

The variable is incremented each iteration until it reaches the value identified as the final state. Our pseudo code syntax will be: -

FOR (starting state, final state, increment)

Statement

Statement

ENDFOR

The loop is controlled by a variable which is a simple count which is assigned a starting value e.g. n:=1. Each pass through the loop involves a series of instructions such as: -

i. compare current value of count to final state. If count has passed the final state (i.e. count > final state) then the loop will terminate, otherwise continue;

ii. add the increment to the count.

Using these terms, the pseudo code definition for a FOR ... ENDFOR loop will be: -

FOR (n:= a, n = b, +1)

<command sequence>

ENDFOR

where a is the starting point, +1 is the increment and b is the final value. This means that the loop will continue to be executed while n is less than or equal to b.

For example: a FOR loop to control an iteration to read a pair of numbers and find the average of each pair where a total of 50 pairs of numbers are to be input would need: -

- a variable to control the loop e.g. count;

- a starting state of count:=1;

- a final value for count, in this case count = 50;

- an increment of 1 (that is 1 would be added to count each pass of the loop, count := count +

Example: -

```
FOR (n:=1, n=3, 1)
   DISPLAY "loop", n
ENDFOR
```

Study Note

Notice how the variable used to control the loop has also been used in the DISPLAY statement. However, if the contents of the variable had been changed by an instruction within the loop this would have interfered with the behavior of the loop. Do not interfere with the contents of variables which are controlling loops and be aware of any other use for the counting variable apart from controlling the FOR loop.

For Example: write a program to calculate the sum and average of a series of numbers. The user will decide how many numbers are entered. The algorithm (solution) for this program is shown as pseudo code and in the form of a flowchart (figure 2.10).

```
Use variables: n, count OF TYPE Integer
               sum, average, number OF TYPE Real
DISPLAY "How many numbers do you want to input?"
ACCEPT count
FOR (n:=1, n = count, +1)
   ACCEPT number
   sum := sum + number
ENDFOR
average:= sum/count
DISPLAY "The sum of the numbers was ", sum
DISPLAY "The average of the numbers was", average
end of program
```

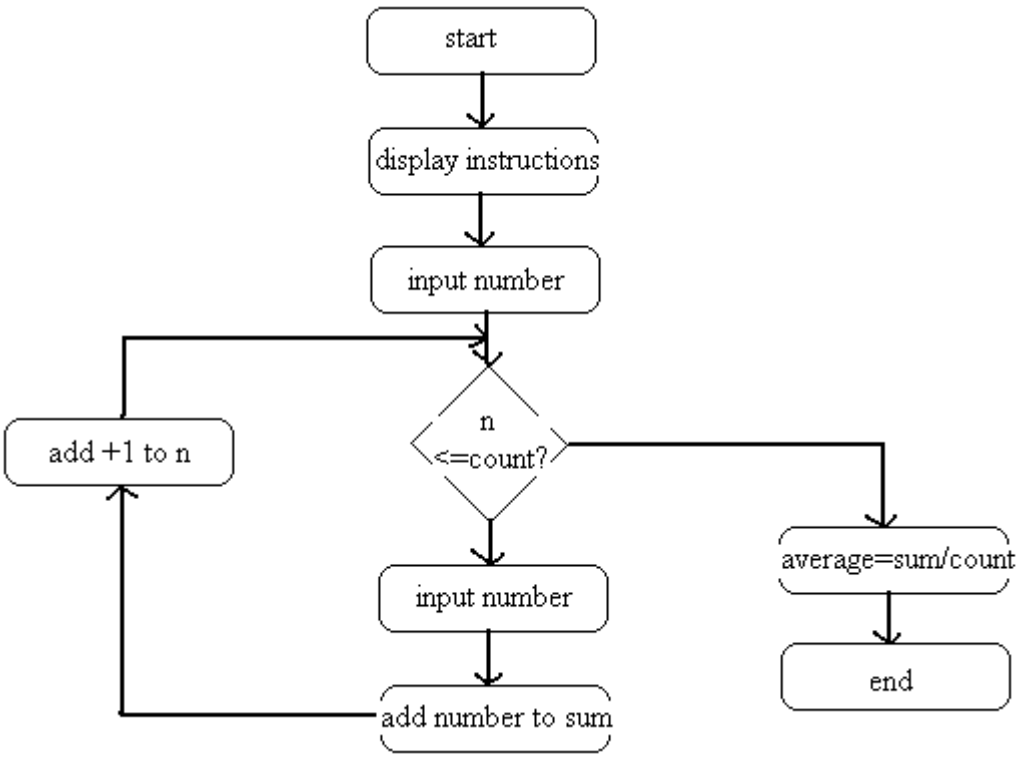


Figure 2.10: Flowchart showing algorithm for program to calculate the sum and average of a series of numbers using the FOR ... ENDFOR statement

Definition: DO loop

A counting loop in a program, in which a section of code is obeyed repeatedly with a counter taking successive values. Thus in FORTRAN,

```
        DO 10 I = 1,100
        <statements>
10      CONTINUE
```

causes the <statements> to be obeyed 100 times. The current value of the counter variable is often used within the loop, especially to index an array. There are many syntactic variants: in Pascal and Algol-related languages the same basic construct appears as the FOR loop, e.g.

```
FOR I := 1 to 100 DO
    BEGIN
        <statements>
```

END

This kind of loop is a constituent of almost all procedural programming languages (except APL, which has array operations defined as operators in the language).

Exercise 2.15

Amend the previous pseudo-code solution to input a student name and marks in exercise 2.12, but this time there are always three examination marks for each student. You are required to provide the average examination mark for each student. Forty five students took the exam. The examination marks input are whole numbers i.e. integers. Use a FOR ... ENDFOR construct.

Exercise 2.16

What type of loop construct would you use to do the following?

a. Perform a series of calculations 10 times.

b. Input a series of numbers terminated with a value of 999999.

c. Calculate the answer to the formula y:= x2 + 2*x for x in the range 100 to 199.

d. Only accept numbers over the keyboard, which are positive, if a number is encountered outside this range then terminate accepting numbers over the keyboard.

e. Calculate the answer to the formula y:= x2 + 2*x for even numbers in the range 100 to 199.